

BENEVOLENTGEMINI

Current Expo Android build - Guide & Showcase

Your Gemini workbench.

Control. Continuity. Portability.

A practical guide to BYOK API use, multi-tab chats, custom model slots, web grounding, image prompts, Read Mode and real document exports.

BYOK

MULTI-TAB

WEB GROUNDING

IMAGE PROMPTS

DOCX + PDF

BenevolentGemini

A focused Gemini workbench for control, continuity and portability.

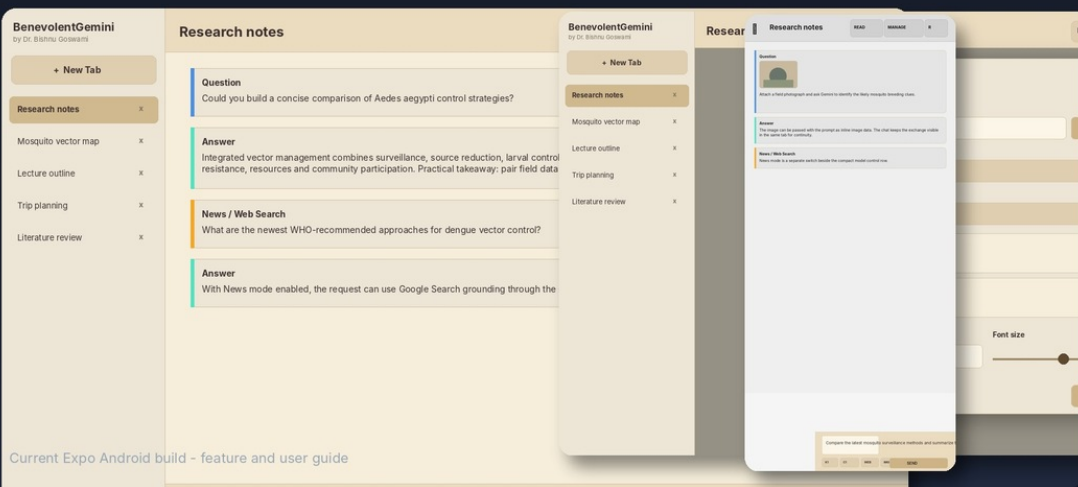
BYOK

MULTI-TAB

WEB GROUNDING

IMAGE PROMPTS

DOCX + PDF



Current Expo Android build - feature and user guide

What this guide covers

- Feature overview and interface map
- Model and API key workflow
- News / Search and image prompts
- Sharing, JSON backups, DOCX and PDF
- Native Gemini comparison and honest boundaries
- Installation, troubleshooting and security notes

Version 1.0.0 | Prepared 30 Sep 2026

00 | GUIDE MAP

Inside this showcase + guide

A visual-first tour of the current build, followed by practical setup, security, troubleshooting and technical notes.

01 What BenevolentGemini is The product story and core architecture of the user experience.	02 Feature map A capability-by-capability overview.
03 The main screen Interface layout and the control row.	04 Conversations Multi-tab chat organization and persistence.
05 API keys BYOK setup, K1/K2 labels and key-store import.	06 Models Built-in choices, C1/C2 custom IDs and quota behavior.
07 News mode Google Search grounding for current-information prompts.	08 Multimodal prompts Image attachment workflow and its current limitation.
09 Manage Share, JSON backup, import and export actions.	10 Output Real OOXML DOCX plus PDF export behavior.
11 Reading Themes, font size and Read Mode.	12 Positioning A fair workflow comparison with native Gemini Apps.
13 Getting started Step-by-step first launch and setup.	14 Security Key handling and trust boundaries.
15 Power-user playbook Repeatable workflows for research, teaching and model experiments.	16 Troubleshooting Common issues and the current code behavior.
17 Under the hood WebView, API and Android native bridge.	18 Honest boundaries What the app does not claim to be.
19 Sources & notes Project basis plus official Google documentation.	

Reading convention: app-specific facts are derived from the supplied project source. Google product notes are attributed to official Google documentation and may change as Gemini evolves.

01 | AT A GLANCE

What BenevolentGemini is

BenevolentGemini is a lightweight Android client around the Gemini API. It keeps the chat experience familiar while adding a layer of explicit control over keys, model IDs, tabs and exports.

Think of it as a personal Gemini workbench rather than another chatbot.

Direct Gemini API client

The app constructs requests in the WebView and sends them directly to the Google Generative Language API using the selected API key and model.

Persistent chat workspace

Chats are kept in local WebView storage under the BenevolentGemini state object, so your tabs and settings can survive app relaunches.

Control surfaces that matter

Key selection, custom model IDs, News/Search mode, image attachment, Read Mode, theme/font settings and practical export/backup tools are exposed close to the chat box.

Portable output

Current or all tabs can be backed up as JSON, shared as plain-text files, or converted into real OOXML DOCX / PDF output from the Manage menu.

02 | FEATURE MAP

What you can actually do

The current build combines a normal chat loop with a surprisingly deep set of controls. The table below separates core chat behavior from power-user features.

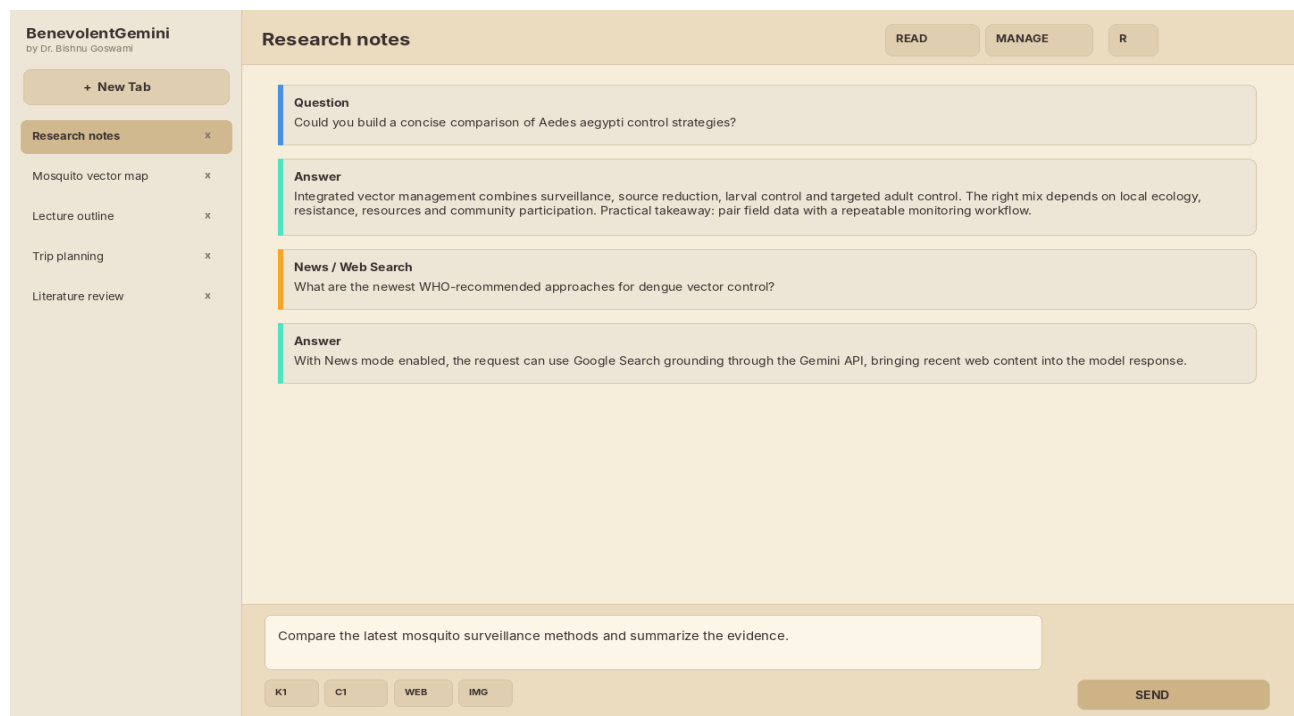
Area	Capability in the current build	Why it matters
Chat organization	Multiple named chat tabs; create, switch and delete; active chat title; persistent state.	Keep research, teaching, planning and one-off questions separated without losing continuity.
API keys	Add multiple Gemini API keys manually or import a .txt key store; keys are shown as K1, K2, K3...	Makes a BYOK workflow practical when you have more than one key or project.
Models	Built-in regular and News mode lists plus arbitrary saved custom model IDs shown as C1, C2, C3...	Lets power users choose the API model explicitly rather than treating the model as invisible plumbing.
Fresh information	News mode adds the Gemini API googleSearch tool to the request.	A simple, visible switch for queries where web-grounded freshness matters.
Images	Attach an image from the device; the image is sent as inline image data with the current request.	Useful for diagrams, screenshots, field photographs and visual questions.
Reading	Read Mode hides the sidebar, topbar and input area; a floating control returns you to normal mode.	Turns a long answer into a cleaner reading surface.
Portability	Share current/all tabs, download current/all tabs, import tab backups, export DOCX Q&A; or answers-only, and export PDF.	Moves useful conversations from the app into files, sharing targets, or a recoverable backup.
Personalization	Warm Sepia, Night Mode, Classic Light, Intelligent Grayscale; font size 12-24 px.	Makes long reading sessions more adaptable to personal preference and environment.

Important boundary: a custom model entry is a saved model ID, not a promise that Google currently exposes that model to your key. Model availability is controlled by the Gemini API and can change independently of the app.

03 | INTERFACE

The main screen: a compact control surface

The UI is deliberately information-dense: chats on the left, the active conversation in the center, and model/key/search/image controls kept close to Send.



Illustrative interface render based on the current app layout and source configuration.

- 1

Chat tabs
The sidebar is a persistent workspace. Create a new chat tab with a key, or select an existing one. It will be used as the stored key, C1/C2... or the built-in labels selects Read Mode, Search, or Image mode.
- 2

Reading width
Conversation content is centered into a readable column width instead of stretching across the whole screen.
- 3

Control row
K1/C1 selects the stored key, C1/C2... or the built-in labels selects Read Mode, Search, or Image mode.
- 4

One-tap modes
Read Mode is deliberately separated from Settings: it is a reading app.

04 | CONVERSATIONS

Chats are treated as workspaces, not just a scrolling history

A new tab starts a fresh conversation. The app gives each chat a short title, keeps the active chat selected, and writes the whole application state to local storage after changes.

Start a new line of thought

Use New Tab before beginning a different topic. This avoids mixing unrelated history into the API context.

Return to interrupted work

Reopen the app and your saved chats can remain available because the application state is persisted locally.

Keep backups separate

Export a current tab when you need a portable snapshot; export all tabs when you want a complete archive.

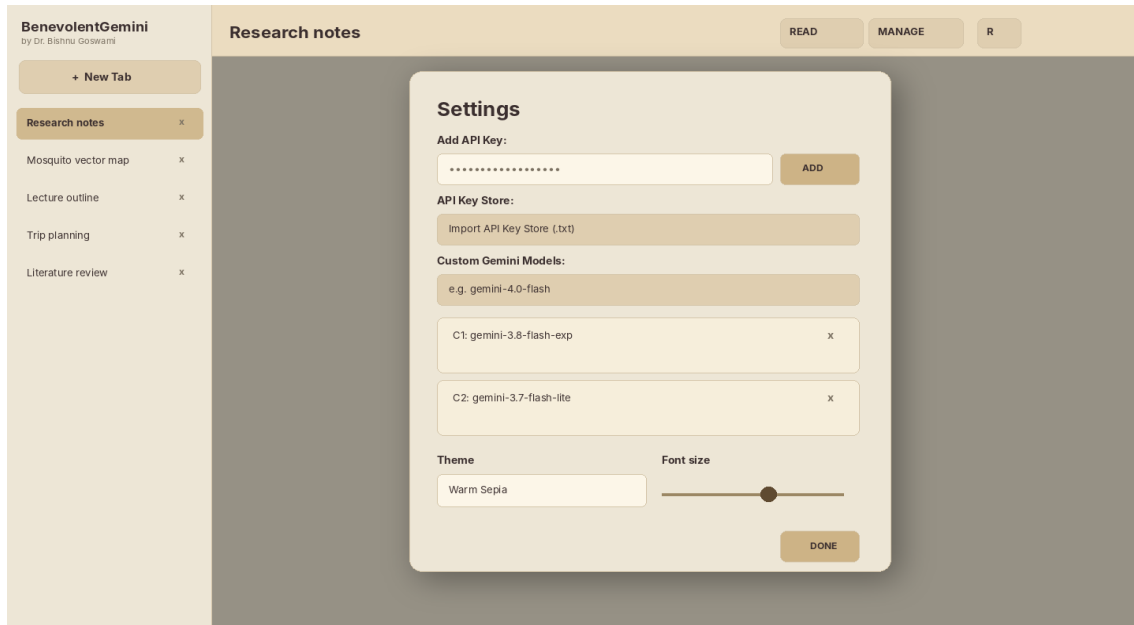
A useful mental model is one tab = one reusable thinking context. Examples: "thesis research", "lecture preparation", "current affairs", "coding help", "travel planning".

Implementation detail: the current build stores chat state in `localStorage` under `benevolentGeminiState`. The Expo wrapper supplies a stable WebView base URL so the storage origin persists between relaunches.

05 | API KEYS

BYOK without turning key management into clutter

The app is designed for people who already use Gemini API keys. Instead of exposing full key strings throughout the interface, it labels them K1, K2, K3... in the chat control row.



The Settings screen includes manual key entry, bulk key-store import, custom model IDs, themes and font size.

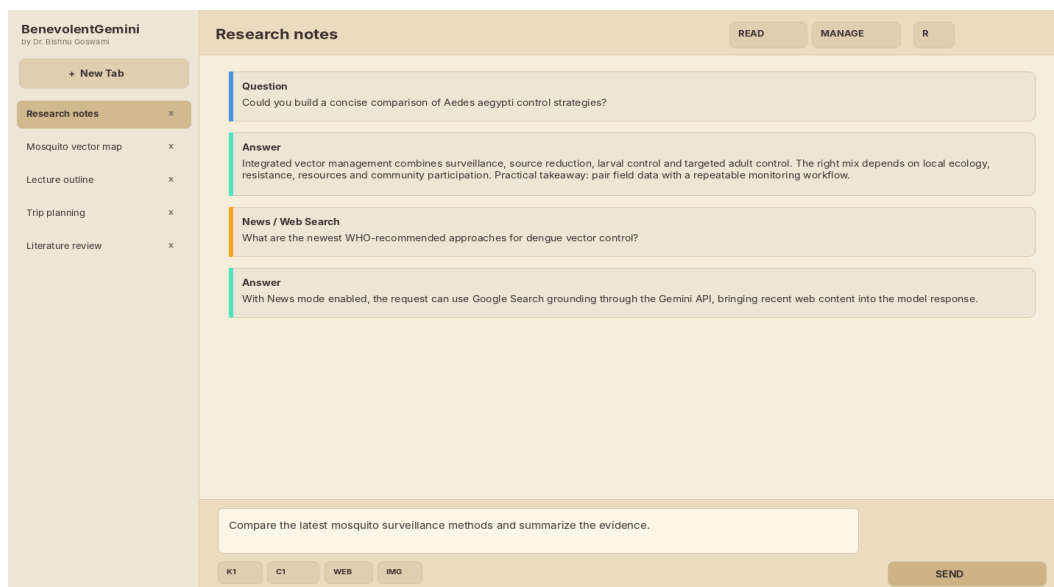
- Manual entry: Paste a Gemini API key into Settings and tap Add. Duplicate keys are not re-added.
- Key Store import: Prepare a plain text file with one key per line. Blank lines are ignored and duplicate keys are skipped.
- Selection: The chat row displays K1, K2... rather than the secret itself. The selected key is used for the next Gemini API request.

Do not confuse two different things: the app's API Key Store is just a plaintext import format. It is not your Android app signing keystore.

06 | MODELS

C1 / C2 custom models + compact built-in model controls

Built-in model choices are shortened visually for the narrow control row. Saved custom model IDs are deliberately abstracted to C1, C2, C3... so the chat bar stays compact.



Built-in model choices

The current build seeds a regular list and a separate News mode list. The exact IDs are part of the app source and may need updating as Google changes model availability.

Custom model choices

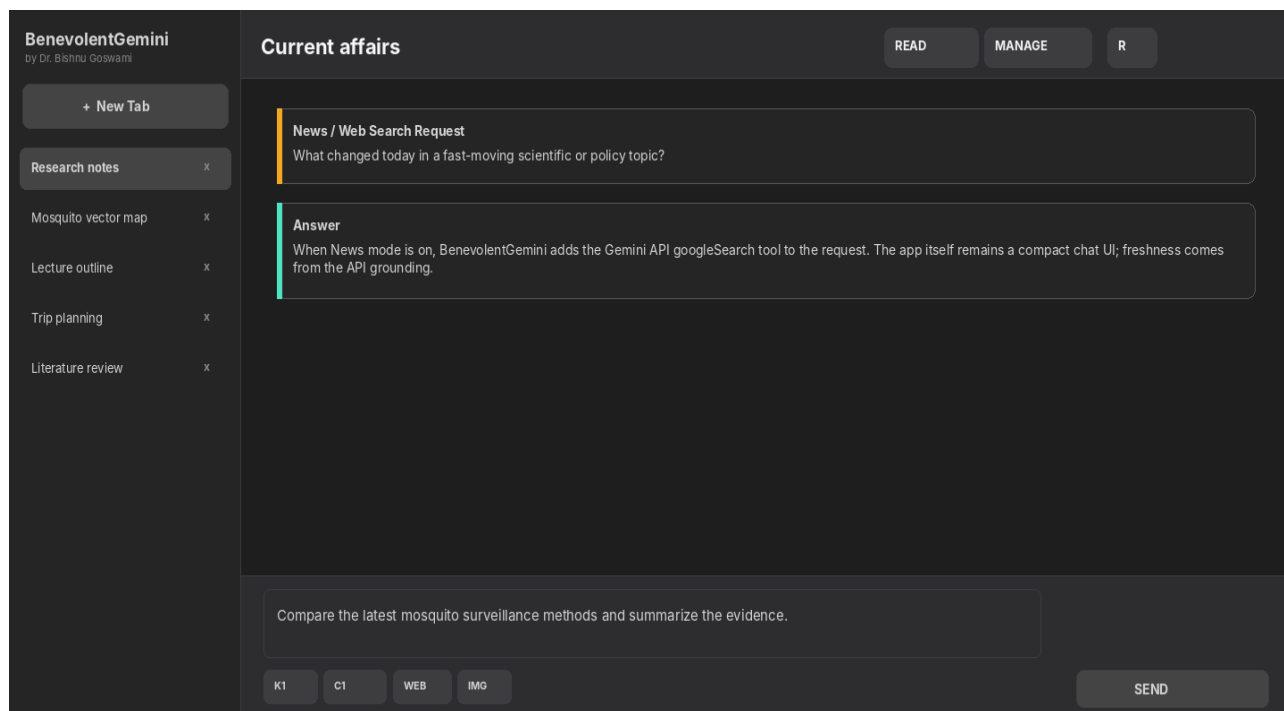
Add any model ID you want to remember. The app preserves the underlying ID while displaying C1, C2... in the main selector.

Quota behavior: in regular mode, when a request fails with a 429/quota/exhausted-style error and another model option exists, the selector advances to the next option and puts the prompt back into the input box. This is a usability fallback, not automatic key rotation.

07 | FRESH INFORMATION

News mode: a visible switch for web-grounded requests

News mode is implemented at the Gemini API request level. When enabled, the payload includes the Google Search tool, so freshness comes from the API grounding rather than from a separate in-app search engine.



Illustrative News/Search workflow render.

- Use it for: recent events, newly released guidance, current product or policy changes, rapidly changing scientific information, and other time-sensitive questions.
- Do not assume: the orange News/Search label by itself guarantees a citation-rich answer. The app enables the tool; what the model returns and how it is rendered still depends on the Gemini API response.
- Current implementation: News mode uses a separate set of model choices from regular mode and sends `tools: [{ googleSearch: {} }]` with the generation request.

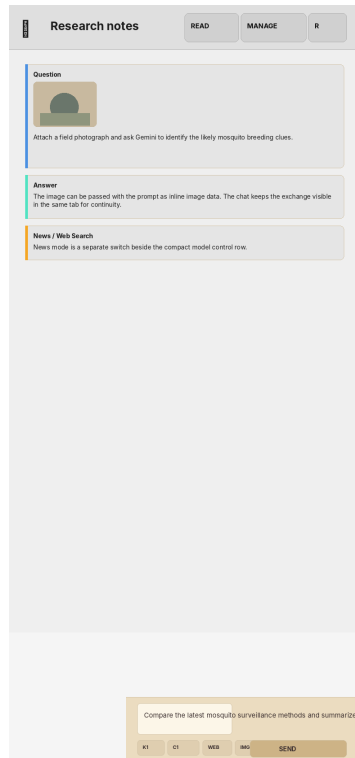
Google reference: Gemini API - Grounding with Google Search (official documentation).

For fairness: the native Gemini product also offers much broader web-connected workflows, including Deep Research. BenevolentGemini is intentionally narrower: it exposes Search grounding as a compact client-side control.

08 | MULTIMODAL PROMPTS

Attach an image when the question is visual

The paperclip action accepts an image file. The app reads the image as data and includes it in the current Gemini request alongside your prompt.



Good uses

- Ask what a screenshot or diagram is showing.
- Describe an object, specimen, chart, UI, label or document fragment.
- Ask for a comparison between visual alternatives.

How it behaves

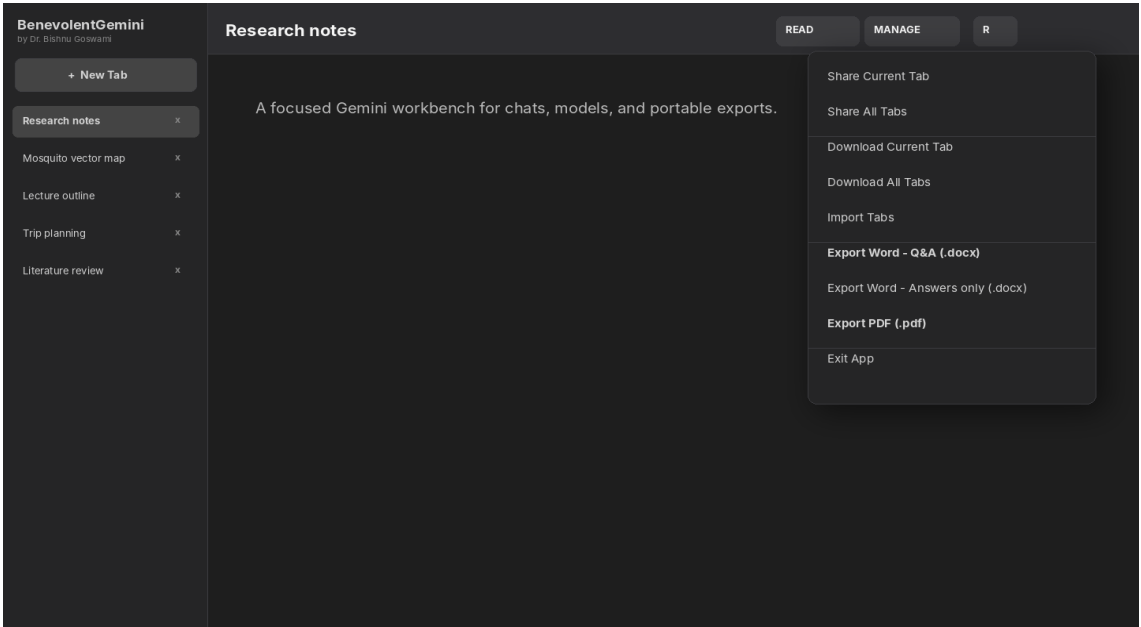
The current image is kept in the chat display and stored with the user message. It is sent as inline image data only for the request in which you attach it.

Important limitation: previous image pixels are not re-sent as image parts when later history is reconstructed. For a follow-up that still depends on the picture, reattach the image.

09 | MANAGE

Portable conversations: share, back up, restore and export

The Manage menu turns chats into files and gives the app a practical escape hatch from the chat UI.



The current Manage menu includes native share/download actions, tab import, Word export, PDF export and Exit App.

Action	Result
Share Current Tab	Creates a text backup file containing the current chat tab and opens the Android share sheet.
Share All Tabs	Creates a text backup file containing all tabs and opens the Android share sheet.
Download Current / All	Creates JSON backup files. On Android, the native layer uses the Storage Access Framework so you can choose a folder; the Downloads folder is suggested first.
Import Tabs	Reads JSON backups produced by the app (and compatible legacy objects) and adds them as new tabs.
Export Word - Q&A;	Creates a real OOXML .docx with Question / Answer labels and page breaks between sets.
Export Word - Answers only	Exports only model answers, without a tab title or Answer heading. Each answer set gets a page break.
Export PDF	Generates a PDF from the chat content. In the Android build, the resulting file is sent to the native share sheet.

10 | OUTPUT

Why the DOCX export is more useful than a renamed web page

The Word export was implemented as actual Office Open XML content. This matters because .docx is a structured Office format, while an HTML blob with a .doc extension is a different thing.

Q&A; mode

The exported document can contain the tab title followed by Question and Answer blocks. Each exchange is separated by a page break.

Answers-only mode

Only model answers are exported. The build intentionally omits both the tab title and an Answer heading in this mode.

Markdown-aware blocks

Headings, lists, emphasis and tables are transformed into document-level Word XML where supported by the current renderer.

Android-friendly transfer

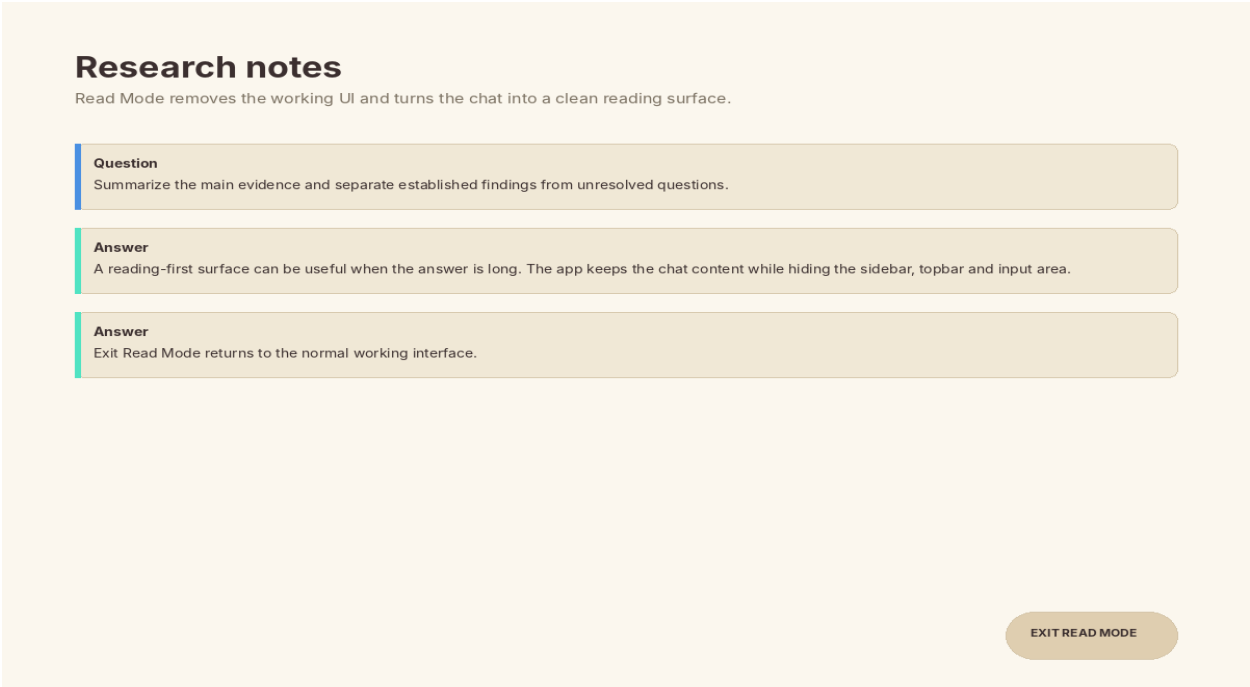
The WebView produces the DOCX blob, streams it in bounded Base64 chunks to the Expo layer, and the native layer shares the finished file.

Practical pattern: use the chat for thinking, then Word for editing, review, printing, submission or archiving. PDF is the more presentation-stable endpoint; DOCX is the more editable endpoint.

11 | READING

Read Mode and personalization

The app includes four themes and a font-size control, plus a dedicated Read Mode that strips the working chrome from the screen.



Illustrative Read Mode render based on the current UI rules.

Setting	Current behavior
Warm Sepia	Default reading-oriented palette with warm background and darker text.
Night Mode	Dark UI with high-contrast message cards and muted chrome.
Classic Light	Bright neutral interface.
Intelligent Grayscale	Reduced-color neutral interface.
Font size	12 to 24 px through the Settings slider.
Read Mode	Hides sidebar, topbar, input and floating prompt button; an exit control brings the full UI back.

12 | POSITIONING

Why it can be useful alongside - or instead of - the native Gemini app

The comparison is about workflow architecture, not intelligence. Google continues to expand the native Gemini Apps experience; BenevolentGemini deliberately optimizes a different layer: explicit API control and portable local chat work.

Dimension	Native Gemini Apps	BenevolentGemini
Core model of use	Google account-centered consumer experience; features such as Deep Research, Gems and Canvas are integrated into the product.	Bring-your-own-key client around the Gemini API.
Model visibility	Model selection is presented through the Gemini Apps product experience and varies with current Google offerings and access.	The app exposes model IDs directly, with compact built-in choices and C1/C2 custom IDs.
Credential control	Account/plan experience is the normal entry point.	You manage your own Gemini API keys, including multiple saved keys and text key-store import.
Chat organization	Integrated Gemini conversation history and feature-specific workspaces.	Explicit local chat tabs designed as separate working contexts.
Search / research	Google offers broad workflows such as Deep Research with Google Search and other connected sources.	A compact News mode turns on Google Search grounding for an API request.
Creation tools	Canvas can create/edit docs, apps, slides and code; Gems add reusable instructions.	Focused chat client with export-oriented output rather than a broader creation suite.
Portability	Google supports share/export workflows for some Gemini outputs; exact options depend on the current feature and account.	Explicit current/all-tab JSON backup, text share, real DOCX export and PDF export are first-class menu actions.
Who benefits most	People who want an integrated Google AI product with account-connected features.	People who value BYOK control, multiple keys, custom model IDs, explicit tab separation and portable local artifacts.

Comparison basis: Google Gemini Apps Help pages for Deep Research, Gems and Canvas, checked 30 Sep 2026.

The positioning in one sentence: native Gemini is the broader product; BenevolentGemini is a narrower, more controllable client experience for people who specifically want the Gemini API as their engine.

13 | GETTING STARTED

First launch: from zero to first answer

The current build expects you to bring a Gemini API key. The app itself does not include a server-side proxy or a shared API credential.

1

Install the APK

Install the Android APK from the release source you trust. Android may ask you to allow installation from your browser or file manager depending on your device settings.

2

Open Settings

Tap the rocket button in the top-right area. The Settings screen contains API key entry, key-store import, custom models, theme and font size.

3

Create a Gemini API key

Use Google AI Studio / the Gemini API key workflow. Google currently documents API key creation and the transition toward authorization keys; choose the current key type and restrictions appropriate to your use.

4

Add one key

Paste the key and tap Add. The chat row should now show a selectable K1 entry.

5

Optional: import a key store

Create a plaintext .txt file with one key per line, then use Import API Key Store. Blank lines and duplicates are ignored.

6

Choose your model

Select a built-in model or save a custom model ID in Settings. Custom entries appear as C1, C2...

7

Send a first prompt

Type a question and tap Send. If you need a visual prompt, attach an image first. If the query needs recent information, turn on News/Search mode.

8

Save the result

Use Manage to share, back up or export the conversation once it is useful enough to keep.

14 | SECURITY

Key handling and trust boundaries

Because this is a BYOK client that calls the Gemini API from the app, the API key is part of the local application state. The strongest marketing claim here is control - not an encrypted vault.

Keys are local state

The current implementation persists the app state, including the saved key list, in WebView localStorage. No Expo SecureStore layer is present in the provided project.

Key Store is plaintext

The import file is intentionally simple: one key per line. Keep that file protected like a password list.

Do not ship a shared secret

For public distribution, do not embed a personal API key into the app and tell every user to share it. Each user should bring their own key.

Follow Google key guidance

Google documents key restrictions and the transition to authorization keys. Use current Google guidance when creating and restricting keys.

Operational rule: never publish a key store, never paste a real API key into screenshots, and revoke a key that may have leaked. Treat keys as credentials, not as app configuration.

What is actually sent: the selected model, your prompt, the conversation text history, and (when attached) the current image data are included in the API request. The app does not run a separate server in the provided project.

15 | POWER-USER PLAYBOOK

Workflows that make the design earn its keep

The real benefit appears when the app becomes a repeatable work surface rather than a place you visit for isolated questions.

Research sprint

Create a tab for the topic -> use News mode for current developments -> ask for a structured evidence map -> keep the tab as the running research log -> export to PDF or DOCX when the working draft is mature.

Teaching preparation

Create one tab per lecture -> attach diagrams or screenshots for visual checks -> use a chosen model consistently -> switch to Read Mode for reviewing the finished explanation -> export answers to DOCX for further editing.

Multiple-key workflow

Import a key store -> select K1/K2 from the chat row depending on the project or quota situation -> keep the key list compact and out of the main reading area.

Model experimentation

Save candidate model IDs as C1/C2/C3 -> run comparable prompts in separate tabs -> export the useful outputs instead of keeping them trapped in one scrolling session.

Long-form reading

Finish the prompt cycle -> tap Read -> adjust font size/theme if needed -> read without sidebar and input chrome -> exit Read Mode to continue working.

16 | TROUBLESHOOTING

Common issues and what the current code actually does

These notes are based on the current app and Expo wrapper, not on generic Android troubleshooting advice.

No API key found

Open Settings. Add a key manually or import the key store. The first-run prompt explicitly points you toward Google AI Studio.

429 / quota error

In regular mode, if another model option exists, the app advances to the next model selector entry and restores the prompt to the input box. It does not automatically rotate through API keys.

News request fails

The app reports that Search grounding failed and restores the prompt. Try another key or turn News mode off.

Image follow-up behaves oddly

The original image is displayed in the chat and saved with the message, but its image bytes are not carried forward into later reconstructed API history. Reattach the image for a later visual follow-up.

Download does not appear in a browser folder

On Android, the wrapper uses the Storage Access Framework for JSON downloads. The user selects a folder; the Downloads folder is suggested first.

DOCX is not .doc

That is intentional. The current build generates a true .docx package using OOXML XML files bundled into a ZIP container.

APK cannot update cleanly after signing-key loss

The signing key establishes Android app identity. Keep a secure backup of the EAS-generated signing credentials for future releases.

17 | UNDER THE HOOD

A simple architecture with a useful native boundary

The current project preserves the HTML/JavaScript application and uses Expo mainly as an Android shell plus a small native bridge for capabilities that browsers handle inconsistently.

USER	BenevolentGemini WebView	Gemini API	Android native bridge
Prompt / image / mode selection	Render chats; persist state; build API payloads; generate exports	GenerateContent request; optional Google Search grounding	Share files; choose download folder; exit app

WebView layer

The main HTML app remains responsible for UI, local state, chat rendering, API calls and file generation.

Stable local origin

The Expo WebView uses a stable base URL and enables DOM storage so localStorage survives app relaunches.

Native file bridge

Exports are streamed from the WebView to Expo in chunks. Sharing uses expo-sharing; downloads use Android Storage Access Framework.

Minimal native surface

The wrapper also fixes the Android status-bar inset and maps Exit App to the Android back handler.

18 | HONEST BOUNDARIES

What the app does not try to be

A credible product story is stronger when its boundaries are explicit.

- Not a new model: BenevolentGemini uses Gemini models through the API. Its differentiator is the client workflow, not a proprietary foundation model.
- Not a general replacement for native Gemini: Google continues to add product-level features such as Deep Research, Gems, Canvas and broader account integrations. BenevolentGemini intentionally focuses on API control and portable chat work.
- Not a server-side privacy proxy: the provided Android project does not contain a backend that sits between the device and Google. The client talks to the API directly.
- Not automatic quota bypass: multiple keys make selection easier; the code does not silently cycle keys to evade quotas. Its regular-mode fallback advances the model selection after certain quota errors.
- Not unlimited multimodal memory: an attached image is kept in the chat display, but the current code does not resend prior image bytes in later history payloads.
- Not a secure key vault: the current state mechanism is localStorage. Use current Google key restrictions and protect your device and key-store files accordingly.

Marketing message you can safely use

"BenevolentGemini turns the Gemini API into a personal, portable workbench - with multi-tab chats, BYOK key control, custom model slots, web-grounded News mode, image prompts, Read Mode and real document exports."

19 | SOURCES & NOTES

Documentation, comparison sources and project basis

App-specific behavior below is based on the current project files. Google product notes were checked against Google's official documentation during preparation on 30 Sep 2026.

Source	Used for
BenevolentGemini Expo project - App.js / appHtml.js / README.md	UI structure, chat state, key-store behavior, custom model slots, News mode, image attachment, exports, Android native bridge and build configuration.
Google AI for Developers - Using Gemini API keys	Current key-authentication guidance, restrictions and the 2026 transition toward authorization keys.
Google AI for Developers - Grounding with Google Search	What Google Search grounding does at the Gemini API level and why it is relevant to News mode.
Gemini Apps Help - Deep Research	Current native Gemini research workflow and the role of Google Search, uploaded files and connected sources.
Gemini Apps Help - Gems	Current native Gemini reusable-instruction workflow and knowledge sources.
Gemini Apps Help - Canvas	Current native Gemini Canvas capabilities for docs, apps, slides and code.

Official references

[Google AI Studio / Gemini API keys](https://ai.google.dev/gemini-api/docs/api-key) - <https://ai.google.dev/gemini-api/docs/api-key>
[Gemini API getting started](https://ai.google.dev/gemini-api/docs/get-started) - <https://ai.google.dev/gemini-api/docs/get-started>
[Grounding with Google Search](https://ai.google.dev/gemini-api/docs/google-search) - <https://ai.google.dev/gemini-api/docs/google-search>
[Gemini Apps: Deep Research](https://support.google.com/gemini/answer/15719111?hl=en) - <https://support.google.com/gemini/answer/15719111?hl=en>
[Gemini Apps: Gems](https://support.google.com/gemini/answer/15146780?hl=en-IN) - <https://support.google.com/gemini/answer/15146780?hl=en-IN>
[Gemini Apps: Canvas](https://support.google.com/gemini/answer/16047321?hl=en-166) - <https://support.google.com/gemini/answer/16047321?hl=en-166>

Version note: this document describes the supplied Expo project version 1.0.0 and its current source code. Model IDs, Google API behavior, and native Gemini features may change independently of the app. Re-verify before publishing release documentation for a later build.

End of guide.